

Hybrid Stochastic-Continuous Models

Coupling ODE dynamics with stochastic rewriting events

Simon Frost

2026-03-30

1 Overview

Many real-world systems combine **continuous dynamics** (growth, decay, diffusion) with **discrete stochastic events** (births, deaths, mutations). AlgebraicABMs supports this naturally through **ABMFlow**, which couples ODE integration with the stochastic rewriting engine.

In a hybrid model:

- **ABMFlow** defines continuous dynamics on ACSet attributes (e.g., population size, concentration)
- **ABMRule** defines stochastic discrete events (e.g., catastrophes, infections)
- The simulation alternates between ODE integration (in steps of `dt`) and stochastic event execution

The key data structures are:

- `RawODE(funcs)`: Wraps a vector of functions `u_i -> du_i/dt`, one per ODE variable
- `ABMFlow(pattern, dynamics, name, conditions, mapping)`: Associates a pattern (with `AttrVar` placeholders) to ODE dynamics

2 Setup

```
using AlgebraicABMs
using AlgebraicABMs.ABMs: RuntimeABM
using Catlab, AlgebraicRewriting
using Distributions: Exponential
using Random
Random.seed!(42)

# Resolve Traj ambiguity (exported by both AlgebraicABMs and AlgebraicRewriting)
const ABMTraj = AlgebraicABMs.ABMs.Traj

AlgebraicABMs.ABMs.Traj
```

3 Example 1: Pure ODE — Linear Growth

We start with a pure continuous model: linear growth of a quantity stored as an attribute.

3.1 Define the Schema

We need a schema with a single object carrying a floating-point attribute.

```
@present SchLSet(FreeSchema) begin
    X::Ob
```

```

D::AttrType
f::Attr(X, D)
end
@acset_type LSet(SchLSet){Float64}
var"##LSet#277"{Float64}

```

3.2 Define the ABMFlow

An ABMFlow requires:

1. **Pattern:** An ACSet with AttrVar placeholders marking which attributes are ODE variables
2. **Dynamics:** A RawODE with one function per variable: $u_i \rightarrow du_i/dt$
3. **Mapping:** Pairs linking attribute types to pattern variable indices

```

# Pattern: single vertex with one attribute variable
v = @acset LSet begin X=1; D=1; f=[AttrVar(1)] end

# Dynamics: constant growth rate of 1.0 per time unit
# Each function takes current value u_i and returns du_i/dt
growth_dynamics = RawODE([_ -> 1.0])

# Create the flow
linear_flow = ABMFlow(v, growth_dynamics, :LinearGrow, [], [(:D => 1)])

println("ABMFlow created: pattern has $(nparts(v, :X)) vertex, $(nparts(v, :D)) AttrVar")
println("Dynamics: du/dt = 1.0 (constant growth)")

ABMFlow created: pattern has 1 vertex, 1 AttrVar
Dynamics: du/dt = 1.0 (constant growth)

```

3.3 Run the pure ODE simulation

With no stochastic rules, we run the ABM with only flows. The `dt` parameter controls the ODE integration step size. Note that pure ODE mode requires the lower-level `RuntimeABM` interface.

```

init = @acset LSet begin X=2; f=[1.0, 2.0] end
abm_ode = ABM(ABMRule[], [linear_flow])

# Create runtime and trajectory objects
rt = RuntimeABM(abm_ode, deepcopy(init))
traj = ABMTraj(deepcopy(init))

# Run with ODE stepping
run!(abm_ode, rt, traj; maxtime=5.0, dt=0.5)

println("Initial values: [1.0, 2.0]")
println("After 5 time units of linear growth (rate=1.0):")
println("  f[1] = $(round(rt.state[:f][1], digits=2)) (expected: 6.0)")
println("  f[2] = $(round(rt.state[:f][2], digits=2)) (expected: 7.0)")

Initial values: [1.0, 2.0]
After 5 time units of linear growth (rate=1.0):
  f[1] = 6.0 (expected: 6.0)
  f[2] = 7.0 (expected: 7.0)

```

3.4 Multiple patches with different initial conditions

Each matching vertex gets its own ODE variable, integrated independently.

```
init3 = @acset LSet begin X=3; f=[0.0, 5.0, 10.0] end
abm_ode3 = ABM(ABMRule[], [linear_flow])
rt3 = RuntimeABM(abm_ode3, deepcopy(init3))
traj3 = ABMTraj(deepcopy(init3))

run!(abm_ode3, rt3, traj3; maxtime=3.0, dt=0.5)

println("Three patches after 3 time units of growth:")
for i in 1:3
    println(" Patch $i: $(round(rt3.state[:f][i], digits=2))")
end

Three patches after 3 time units of growth:
Patch 1: 3.0
Patch 2: 8.0
Patch 3: 13.0
```

4 Example 2: Hybrid — Growth + Stochastic Duplication

Now we combine continuous growth with a stochastic event that duplicates a vertex (copying its attribute value).

```
# Duplication rule: one vertex becomes two (same attribute value)
v1 = @acset LSet begin X=1; D=1; f=[AttrVar(1)] end
v2 = @acset LSet begin X=2; D=1; f=[AttrVar(1), AttrVar(1)] end

dup_rule = ABMRule(:Duplicate,
    Rule(id(v1), homomorphism(v1, v2; initial=(X=[1],))),
    DiscreteHazard(2.0)) # fires at t=2 from enablement

println("Duplicate rule: vertex splits into two (copies attribute)")

Duplicate rule: vertex splits into two (copies attribute)

init_h = @acset LSet begin X=2; f=[1.0, 2.0] end
abm_hybrid = ABM([dup_rule], [linear_flow])

rt_h = RuntimeABM(abm_hybrid, deepcopy(init_h))
traj_h = ABMTraj(deepcopy(init_h))

run!(abm_hybrid, rt_h, traj_h; maxtime=3.0, dt=0.1)

println("Hybrid simulation (growth + duplication at t=2):")
println(" Initial: 2 vertices, f=[1.0, 2.0]")
println(" Final: $(nparts(rt_h.state, :X)) vertices")
println(" Attribute values: $(round.(rt_h.state[:f], digits=2))")
println(" Events fired: $(length(traj_h.events))")

Hybrid simulation (growth + duplication at t=2):
Initial: 2 vertices, f=[1.0, 2.0]
Final: 4 vertices
Attribute values: [5.0, 5.0, 4.0, 4.0]
Events fired: 2
```

4.1 Understanding the interplay

In the hybrid model, between stochastic events the ODE continuously integrates. When a duplication occurs, the ODE system is rebuilt to account for the new topology.

```
if !isempty(traj_h.events)
    println("Event timeline:")
    for (t, _, name, _) in traj_h.events
        println("  t=$(round(t, digits=3)): $name")
    end
end
```

```
Event timeline:
  t=2.0: Duplicate
  t=2.0: Duplicate
```

5 Example 3: Growth with Stochastic Removal

We can also combine continuous attribute growth with stochastic vertex removal.

```
# Removal rule: delete a vertex
v_del = @acset LSet begin X=1; D=1; f=[AttrVar(1)] end
empty = @acset LSet begin end

remove_rule = ABMRule(:Remove,
    Rule(homomorphism(empty, v_del), id(empty)),
    ContinuousHazard(0.5))

println("Remove rule: delete a vertex at rate 0.5")

Remove rule: delete a vertex at rate 0.5

init_r = @acset LSet begin X=5; f=[1.0, 2.0, 3.0, 4.0, 5.0] end
abm_growth_death = ABM([remove_rule], [linear_flow])

rt_r = RuntimeABM(abm_growth_death, deepcopy(init_r))
traj_r = ABMTraj(deepcopy(init_r))

run!(abm_growth_death, rt_r, traj_r; maxtime=5.0, dt=0.5)

println("Growth + removal simulation:")
println("  Initial: 5 vertices")
println("  Final: $(nparts(rt_r.state, :X)) vertices")
if nparts(rt_r.state, :X) > 0
    println("  Surviving values: $(round.(rt_r.state[:f], digits=2))")
end
println("  Events: $(length(traj_r.events))")

Growth + removal simulation:
  Initial: 5 vertices
  Final: 0 vertices
  Events: 5
```

6 Example 4: Sensitivity to ODE Step Size

The `dt` parameter controls the trade-off between accuracy and computational cost.

```
println("Effect of dt on ODE accuracy (10 time units, growth rate 1.0):")
for dt in [2.0, 1.0, 0.5, 0.1]
    init_dt = @acset LSet begin X=1; f=[0.0] end
    abm_dt = ABM(ABMRule[], [linear_flow])
    rt_dt = RuntimeABM(abm_dt, deepcopy(init_dt))
    traj_dt = ABMTraj(deepcopy(init_dt))
    run!(abm_dt, rt_dt, traj_dt; maxtime=10.0, dt=dt)
    val = round(rt_dt.state[:f][1], digits=4)
    err = round(abs(val - 10.0), digits=4)
    println(" dt=$dt: f=$(val), error=$(err)")
end
```

```
Effect of dt on ODE accuracy (10 time units, growth rate 1.0):
dt=2.0: f=10.0, error=0.0
dt=1.0: f=10.0, error=0.0
dt=0.5: f=10.0, error=0.0
dt=0.1: f=10.0, error=0.0
```

7 Summary

Component	Purpose	Key Type
RawODE([f1, f2, ...])	ODE dynamics functions $u_i \rightarrow du_i/dt$	AbsDynamics
ABMFlow(pat, dyn, name, acs, mapping)	Associates pattern to continuous dynamics	Struct
AttrVar(i)	Placeholder in pattern for i-th ODE variable	Pattern construction
mapping = [(:D => 1)]	Links attribute type :D to variable index 1	Flow configuration
dt kwarg in run!	ODE integration step size	Simulation parameter
RuntimeABM(abm, init)	Low-level runtime for hybrid simulation	Runtime state

Key points:

- Each ABMFlow pattern with AttrVar placeholders creates ODE variables
- Multiple matches of the same pattern create independent ODE variables
- Stochastic events can change topology; the ODE system is rebuilt afterward
- The dt parameter controls how often ODE integration steps are taken between checking for stochastic events